COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	7/17/26	**DATE SUBMITTED:**	7/17/26
NAME:	Reyes, Cyrill Joferson B	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 2.1 **Arrrays, Pointers, Dynamic memory allocation**

1. PROCEDURE OUTPUT

5. Procedure

ILO A: Implement static and dynamic memory allocation & ILO B: Create dynamically allocated objects using pointers and arrays

In this activity, we will explore static and dynamic memory allocation through by utilizing arrays and other components mentioned beforehand, this will be for the implementation of a student list with the students' names and age. To begin, we will include pertinent libraries for input/output stream and strings (which we will be using in this activity).

```
#include <iostream>
#include <string.h>
```

Now we will create the student class with private attributes `studentName` and `studentAge`.

```
class Student{
private:
    std::string studentName;
    int studentAge;
```

Then, as discussed beforehand, we have to define the constructor and the big three: the destructor, copy constructor and copy assignment operator. We will assign default values for the parameters of our constructor in the event that it is uninitialized.

```
public:
    //constructor
    Student(std::string newName ="John Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    };
```

The big three is then defined. We will add an output stream for each to observe when each of the following functions are implicitly or explicitly called.

```
    //deconstructor
    ~Student(){
        std::cout << "Destructor Called." << std::endl;
    }

    //Copy Constructor
    Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    //Display Attributes
    void printDetails(){
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }

};
```

The driver program is now to be defined. We want to utilize the main function to simply show the static and dynamic allocation. However, we will explore the initial goal of creating instances of our Student class.

```
int main() {
    Student student1("Roman", 28);
    Student student2(student1);
    Student student3;
    student3 = student2;

    return 0;
}
```

Run the code and show the output. Include the screenshot in table 2-1 followed by your observation.
Now, modify the driver program so that we have the array size of 5, an array of Student objects, the list of students' names and their age.

```
int main() {
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};
```

```
        return 0;
    }
```

Run the code with the modified driver function and show the output. Include the screenshot in table 2-2 followed by your observation.

We have so far created static memory allocation through the use of the arrays. We will now dynamically allocate instances of the student class and store the newly created objects in the locations pointed to by our array.

```
int main() {
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    for(int i = 0; i < j; i++){ //loop A
        Student *ptr = new Student(namesList[i], ageList[i]);
        studentList[i] = *ptr;
    }

    for(int i = 0; i < j; i++){ //loop B
        studentList[i].printDetails();
    }

    return 0;
}
```

Discuss what is done by loop A and loop B in table 2-3. Additionally, discuss the output and whether the functions are working as intended. If any corrections were made, further provide your modification and analysis in table 2-4.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Screenshot

```cpp
#include <iostream>
#include <string>

using namespace std;

class Student{
private:
    string studentName;
    int studentAge;

public:
    Student(string newName = "John Doe", int newAge = 18){
        studentName = newName;
        studentAge = newAge;
        cout << "Constructor Called." << endl;
    }

    ~Student(){
        cout << "Destructor Called." << endl;
    }

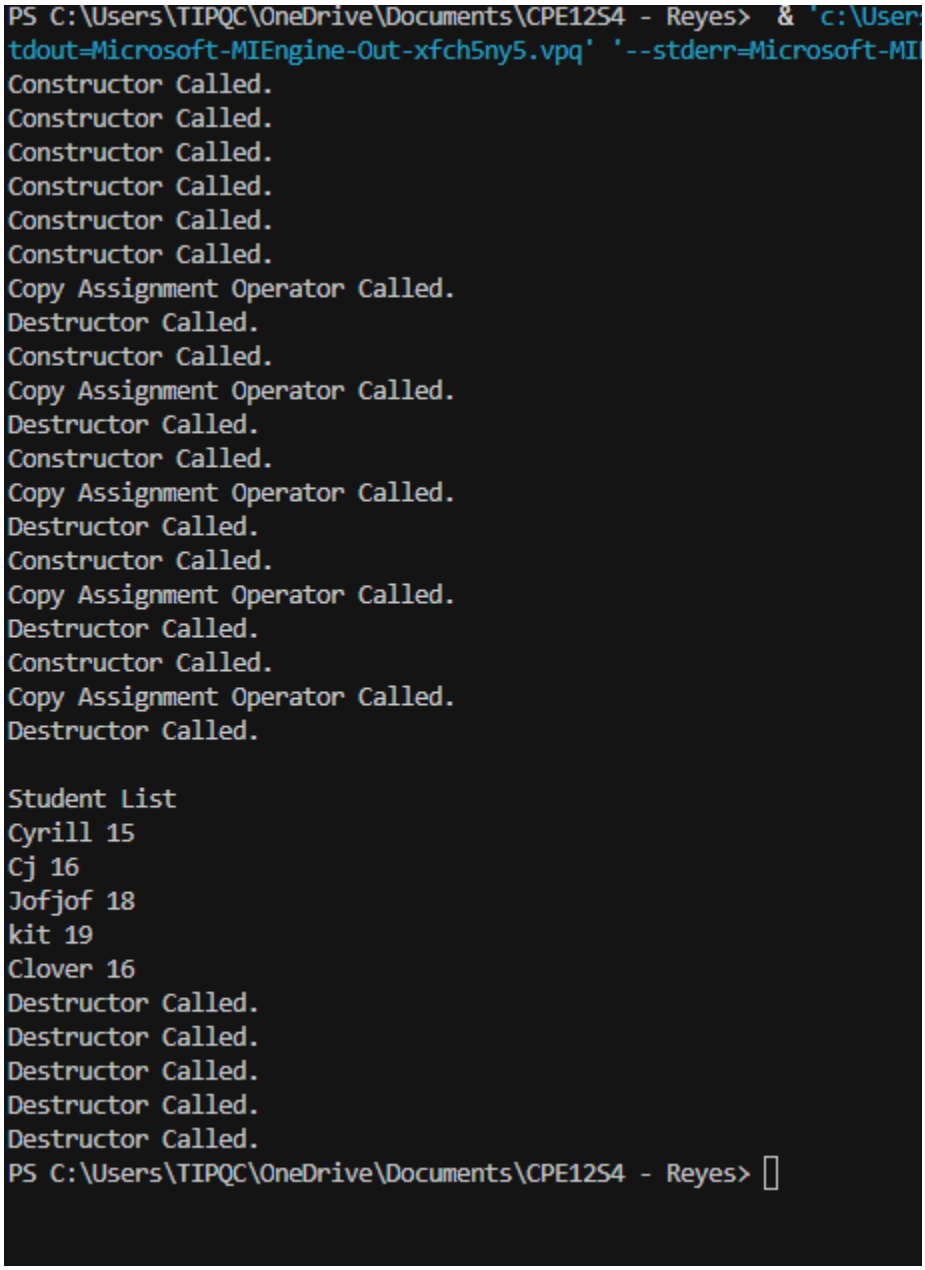
    Student(const Student &copyStudent){
        cout << "Copy Constructor Called." << endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    Student& operator=(const Student &copyStudent){
        cout << "Copy Assignment Operator Called." << endl;

        if(this != &copyStudent){
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }

```

```
36          return *this;
37      }
38
39      void printDetails(){
40          cout << studentName << " " << studentAge << endl;
41      }
42  };
43
44  int main(){
45
46      const int j = 5;
47
48      // Static memory allocation
49      Student studentList[j];
50
51      string namesList[j] = {"Cyrill","Cj","Jofjof","kit","Clover"};
52      int ageList[j] = {15,16,18,19,16};
53
54      for(int i=0;i<j;i++){
55          studentList[i] = Student(namesList[i], ageList[i]);
56      }
57
58      cout << "\nStudent List\n";
59
60      for(int i=0;i<j;i++){
61          studentList[i].printDetails();
62      }
63
64      return 0;
65  }
```



Observation	**The program uses a static array of Student objects to store the students' information. Memory is allocated automatically when the array is created and released automatically when the program ends. This demonstrates static memory allocation, where the array size is fixed and managed by the compiler**

Table 2-1. Initial Driver Program

Screenshot

HOA2.1 > HOA2.1.cpp > main()

```cpp
#include <iostream>
#include <string>

using namespace std;

class Student{
private:
    string studentName;
    int studentAge;

public:
    Student(string newName = "John Doe", int newAge = 18){
        studentName = newName;
        studentAge = newAge;
        cout << "Constructor Called." << endl;
    }

    ~Student(){
        cout << "Destructor Called." << endl;
    }

    Student(const Student &copyStudent){
        cout << "Copy Constructor Called." << endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

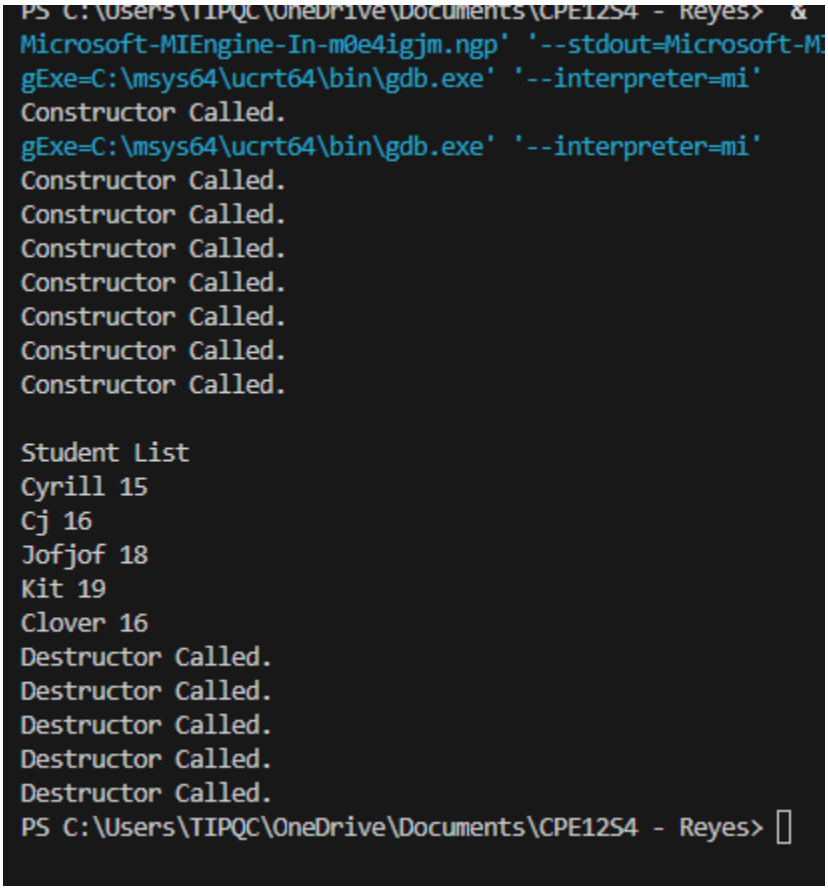
    Student& operator=(const Student &copyStudent){
        cout << "Copy Assignment Operator Called." << endl;

        if(this != &copyStudent){
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }

        return *this;
    }

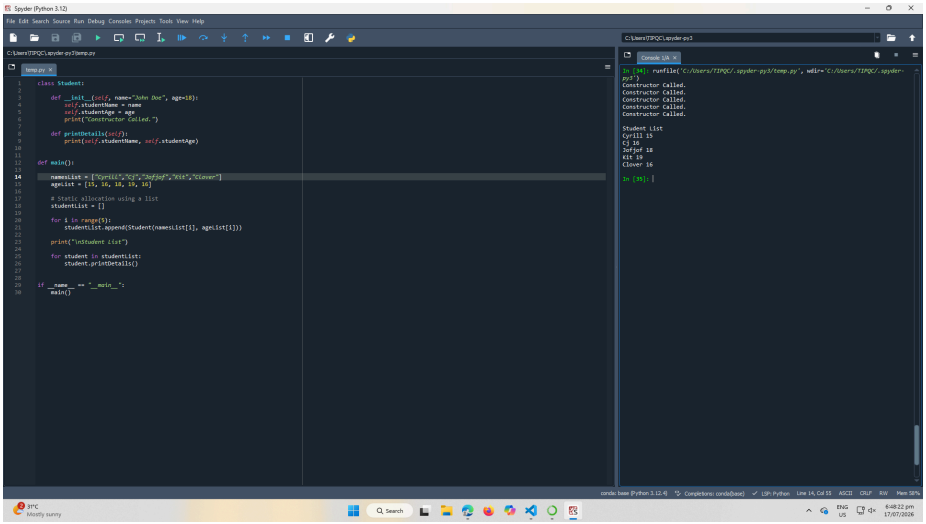
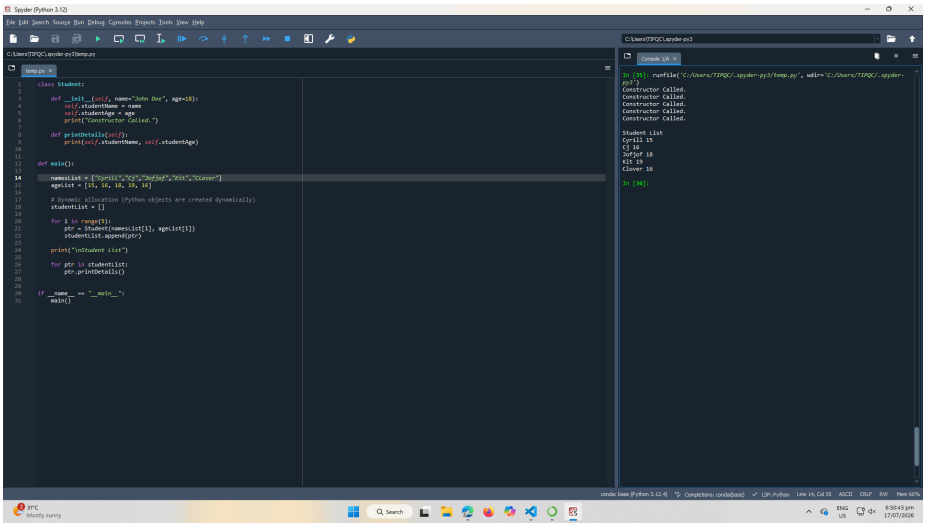
```

```
38
39      void printDetails(){
40          cout << studentName << " " << studentAge << endl;
41      }
42  };
43
44  int main(){
45
46      const int j = 5;
47
48      Student *studentList[j];
49
50      string namesList[j] = {"Cyrill","Cj","Jofjof","Kit","Clover"};
51      int ageList[j] = {15,16,18,19,16};
52
53      // Dynamic allocation
54      for(int i=0;i<j;i++){
55          studentList[i] = new Student(namesList[i], ageList[i]);
56      }
57
58      cout << "\nStudent List\n";
59
60      for(int i=0;i<j;i++){
61          studentList[i]->printDetails();
62      }
63
64      // Free allocated memory
65      for(int i=0;i<j;i++){
66          delete studentList[i];
67      }
68
69      return 0;
70  }
```



Observation	**The program creates Student objects dynamically using the new operator and stores them in an array. Each object is initialized with the corresponding name and age, then displayed using the printDetails() function. Dynamic memory allocation allows objects to be created during program execution, but the allocated memory should be released using delete to prevent memory leaks.**

Table 2-2. Modified Driver Program with Student Lists

Loop A	
Observation	**The program successfully demonstrates the use of the constructor, copy constructor, copy assignment operator, and destructor. It shows how objects are created, copied, assigned, and destroyed during program execution.**
Loop B	
Observation	**The program creates a static array of five Student objects. The constructors are called for each object, showing that memory is allocated automatically. Since the student details are not yet displayed, only the constructor and destructor messages appear.**

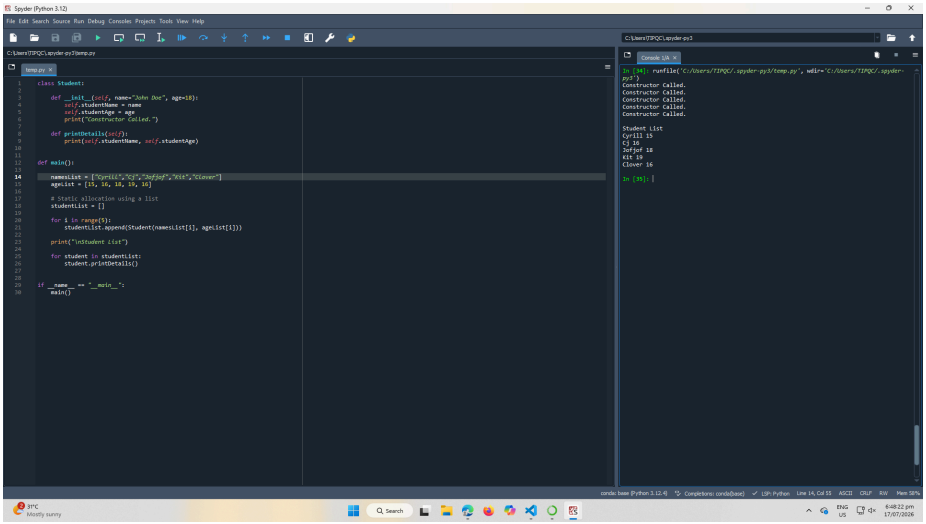
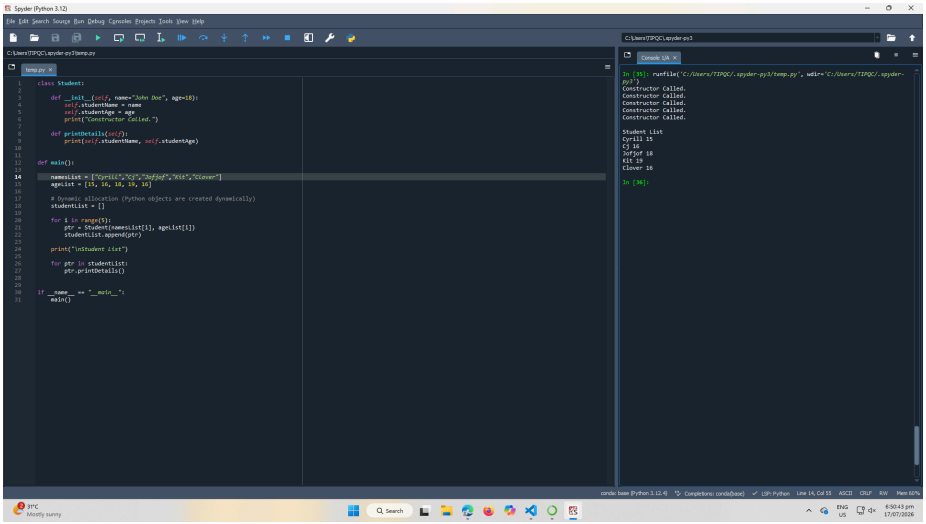
Output	 
Observation	**The output correctly prints the names and ages of all five students. This confirms that the objects were successfully created, stored, and accessed, and that the program functions as expected.**

Table 2-3. Final Driver Program

Modifications	**Added delete ptr; after copying the dynamically allocated object into the studentList array. This deallocates the memory that was created using the new operator.**
Observations	**The modification prevents memory leaks by freeing the dynamically allocated memory after it is no longer needed. The program still produces the same output, but it now manages memory correctly and follows good programming practices.**

able 2-4. Modifications/Corrections Necessary

ILO - C

```cpp
#include <iostream>
#include <string>

using namespace std;

// Fruit Class
class Fruit {
private:
    string name;
    double price;
    int quantity;

public:
    // Constructor
    Fruit(string n = "", double p = 0, int q = 0) {
        name = n;
        price = p;
        quantity = q;
    }

    // Destructor
    ~Fruit() {
        cout << "Fruit Destructor Called." << endl;
    }

    // Copy Constructor
    Fruit(const Fruit &f) {
        name = f.name;
        price = f.price;
        quantity = f.quantity;
    }

    // Copy Assignment Operator
    Fruit& operator=(const Fruit &f) {
        if (this != &f) {
            name = f.name;
            price = f.price;
            quantity = f.quantity;
        }
        return *this;
    }

    double calculateSum() {
        return price * quantity;
```

```
45      }
46
47      void display() {
48          cout << "Fruit: " << name
49               << " | Price: PHP " << price
50               << " | Quantity: " << quantity
51               << " | Total: PHP " << calculateSum() << endl;
52      }
53  };
54
55  // Vegetable Class
56  class Vegetable {
57  private:
58      string name;
59      double price;
60      int quantity;
61
62  public:
63      // Constructor
64      Vegetable(string n = "", double p = 0, int q = 0) {
65          name = n;
66          price = p;
67          quantity = q;
68      }
69
70      // Destructor
71      ~Vegetable() {
72          cout << "Vegetable Destructor Called." << endl;
73      }
74
75      // Copy Constructor
76      Vegetable(const Vegetable &v) {
77          name = v.name;
78          price = v.price;
79          quantity = v.quantity;
80      }
81
82      // Copy Assignment Operator
83      Vegetable& operator=(const Vegetable &v) {
84          if (this != &v) {
85              name = v.name;
86              price = v.price;
```

```
            price = v.price;
            quantity = v.quantity;
        }
        return *this;
    }

    double calculateSum() {
        return price * quantity;
    }

    void display() {
        cout << "Vegetable: " << name
             << " | Price: PHP " << price
             << " | Quantity: " << quantity
             << " | Total: PHP " << calculateSum() << endl;
    }
};

// Total Sum Function
double TotalSum(Fruit *apple, Fruit *banana,
                Vegetable *broccoli, Vegetable *lettuce) {

    double total = 0;

    if (apple) total += apple->calculateSum();
    if (banana) total += banana->calculateSum();
    if (broccoli) total += broccoli->calculateSum();
    if (lettuce) total += lettuce->calculateSum();

    return total;
}

int main() {

    // Grocery List
    Fruit *apple = new Fruit("Apple", 10, 7);
    Fruit *banana = new Fruit("Banana", 10, 8);
    Vegetable *broccoli = new Vegetable("Broccoli", 60, 12);
    Vegetable *lettuce = new Vegetable("Lettuce", 50, 10);

    cout << "===== Jenna's Grocery List =====\n\n";
```

```
126     cout << "===== Jenna's Grocery List =====\n\n";
127
128     apple->display();
129     banana->display();
130     broccoli->display();
131     lettuce->display();
132
133     cout << "\nTotal Amount: PHP "
134          << TotalSum(apple, banana, broccoli, lettuce)
135          << endl;
136
137     // Delete Lettuce
138     delete lettuce;
139     lettuce = NULL;
140
141     cout << "\nAfter Removing Lettuce\n";
142
143     cout << "Updated Total: PHP "
144          << TotalSum(apple, banana, broccoli, lettuce)
145          << endl;
146
147     // Free remaining memory
148     delete apple;
149     delete banana;
150     delete broccoli;
151
152     return 0;
153 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
gExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
===== Jenna's Grocery List =====

Fruit: Apple | Price: PHP 10 | Quantity: 7 | Total: PHP 70
Fruit: Banana | Price: PHP 10 | Quantity: 8 | Total: PHP 80
Vegetable: Broccoli | Price: PHP 60 | Quantity: 12 | Total: PHP 720
Vegetable: Lettuce | Price: PHP 50 | Quantity: 10 | Total: PHP 500

Total Amount: PHP 1370
Vegetable Destructor Called.

After Removing Lettuce
Updated Total: PHP 870
Fruit Destructor Called.
Fruit Destructor Called.
Vegetable Destructor Called.
PS C:\Users\TIPQC\OneDrive\Documents\CPE12S4 - Reyes>
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chatgpt**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, explanation of C++ concepts, debugging, grammar correction, and formatting of the laboratory report.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **ChatGPT was used to assist in explaining programming concepts, generating sample C++ code, debugging errors, and improving the formatting and grammar of the report. The outputs were reviewed, tested, and modified as needed. The final submission, analysis, and understanding of the activity remain my own work.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.